```
"""SULTAN WAVE — সিগন্যাল বট (শুধু সিগন্যাল, ট্রেড আপনি নিজে নেবেন)
GitHub Actions-এ বিনামূল্যে প্রতি ~৫ মিনিটে একবার চলে।"""
import os, json, sys
from datetime import datetime, timezone
import requests
import pandas as pd

TOKEN = os.environ.get("BOT_TOKEN", "")
API = f"https://api.telegram.org/bot{TOKEN}"
STATE_FILE = "state.json"

# ===== সেটিংস (দরকারে বদলান) =====
BALANCE = 100.0          # আপনার এক্সচেঞ্জের ব্যালেন্স ($), শুধু পজিশন সাইজ দেখানোর জন্য
RISK_USD = 2.0           # প্রতি ট্রেডে ঝুঁকি ($)
TARGET_USD = 5.5         # প্রতি ট্রেডে লাভের লক্ষ্য ($)
RR = TARGET_USD / RISK_USD
ATR_SL_MULT = 1.5        # SL দূরত্ব = 1.5 x ATR
SIGNAL_LIFE_H = 12       # এত ঘণ্টায় SL/TP না ছুঁলে সিগন্যাল বাতিল
SYMBOLS = {"BTCUSDT": "BTC/USDT", "PAXGUSDT": "GOLD (PAXG টোকেন, XAUUSD-এর কাছাকাছি
HOSTS = ["https://data-api.binance.vision", "https://api.binance.com"]

# ===== Telegram =====
def tg(method, **params):
    r = requests.post(f"{API}/{method}", json=params, timeout=20)
    return r.json()

def send(chat, text, kb=None):
    p = {"chat_id": chat, "text": text}
    if kb:
        p["reply_markup"] = kb
    tg("sendMessage", **p)

KEYBOARD = {"keyboard": [[{"text": "/status"}, {"text": "/open"}], [{"text": "/st
            "resize_keyboard": True}

# ===== স্টেট =====
def load():
    if os.path.exists(STATE_FILE):
        try:
            return json.load(open(STATE_FILE))
        except Exception:
            pass
    return {"owner": None, "chat": None, "offset": 0, "paused": False, "open": []
```

```
            "history": [], "last_candle": {}, "next_id": 1}

def save(S):
    S["history"] = S["history"][-300:]
    json.dump(S, open(STATE_FILE, "w"), indent=1)

# ===== মার্কেট ডেটা =====
def klines(symbol, interval, limit=300, start=None):
    p = {"symbol": symbol, "interval": interval, "limit": limit}
    if start:
        p["startTime"] = int(start)
    err = None
    for h in HOSTS:
        try:
            r = requests.get(h + "/api/v3/klines", params=p, timeout=15)
            r.raise_for_status()
            df = pd.DataFrame(r.json(), columns=["t", "o", "h", "l", "c", "v", "c
            for c in "ohlc":
                df[c] = df[c].astype(float)
            return df
        except Exception as e:
            err = e
    raise RuntimeError(f"{symbol} ডেটা আনা যায়নি: {err}")

def price(symbol):
    for h in HOSTS:
        try:
            r = requests.get(h + "/api/v3/ticker/price", params={"symbol": symbol
            r.raise_for_status()
            return float(r.json()["price"])
        except Exception:
            pass
    raise RuntimeError("দাম আনা যায়নি")

def ema(s, n): return s.ewm(span=n, adjust=False).mean()

def rsi(s, n=14):
    d = s.diff()
    up = d.clip(lower=0).ewm(alpha=1 / n, adjust=False).mean()
    dn = (-d.clip(upper=0)).ewm(alpha=1 / n, adjust=False).mean()
    return 100 - 100 / (1 + up / dn.replace(0, 1e-12))

def atr(df, n=14):
    pc = df["c"].shift()
    tr = pd.concat([df["h"] - df["l"], (df["h"] - pc).abs(), (df["l"] - pc).abs()
    return tr.ewm(alpha=1 / n, adjust=False).mean()
```

```
def fp(x):
    return f"{x:,.2f}" if x >= 1 else f"{x:.5f}"

# ===== স্ট্র্যাটেজি: 1H ট্রেন্ড + 15M পুলব্যাক =====
def analyze(symbol):
    h1 = klines(symbol, "1h", 250)
    h1["e50"], h1["e200"] = ema(h1["c"], 50), ema(h1["c"], 200)
    x = h1.iloc[-2]
    trend = "UP" if (x.e50 > x.e200 and x.c > x.e50) else "DOWN" if (x.e50 < x.e2
    m = klines(symbol, "15m", 120)
    m["e21"], m["rsi"], m["atr"] = ema(m["c"], 21), rsi(m["c"]), atr(m)
    k = m.iloc[-2]
    side = None
    if trend == "UP" and k.l <= k.e21 < k.c and k.c > k.o and 40 <= k.rsi <= 65:
        side = "BUY"
    elif trend == "DOWN" and k.h >= k.e21 > k.c and k.c < k.o and 35 <= k.rsi <=
        side = "SELL"
    return {"symbol": symbol, "trend": trend, "side": side, "rsi": float(k.rsi),
            "atr": float(k.atr), "candle": int(k.t)}

def make_signal(a):
    entry = price(a["symbol"])
    sl_d = ATR_SL_MULT * a["atr"]
    d = 1 if a["side"] == "BUY" else -1
    return {"symbol": a["symbol"], "side": a["side"], "entry": entry,
            "sl": entry - d * sl_d, "tp": entry + d * sl_d * RR, "sl_d": sl_d,
            "opened": int(datetime.now(timezone.utc).timestamp() * 1000), "trend"

def signal_text(s, title="🔔 নতুন সিগন্যাল"):
    name = SYMBOLS[s["symbol"]].split(" ")[0].split("/")[0]
    qty = RISK_USD / s["sl_d"]
    notional = qty * s["entry"]
    sl_pct = s["sl_d"] / s["entry"] * 100
    tp_pct = sl_pct * RR
    d = 1 if s["side"] == "BUY" else -1
    return (f"{title} #{s.get('id', '')}\n"
            f"{'🟢 BUY (লং)' if d == 1 else '🔴 SELL (শর্ট)'} — {SYMBOLS[s['symbol'
            f"👉 এন্ট্রি: {fp(s['entry'])} (এখনকার দামের কাছাকাছি)\n"
            f"🛑 স্টপ লস: {fp(s['sl'])} ({-sl_pct:.2f}%)\n"
            f"🎯 টেক প্রফিট: {fp(s['tp'])} (+{tp_pct:.2f}%)\n\n"
            f"ঝুঁকি ${RISK_USD:.2f} → লাভ ${TARGET_USD:.2f} (১:{RR:.2f})\n"
            f"পজিশন সাইজ ≈ {qty:.5f} {name} (≈ ${notional:,.0f}), "
            f"লিভারেজ ≈ {notional / BALANCE:.1f}x (ব্যালেন্স ${BALANCE:.0f} ধরে)\n\n"
            f"কারণ: 1H ট্রেন্ড {s['trend']}, 15M-এ EMA21 থেকে পুলব্যাক শেষে ফিরেছে, RSI {s['
            f"⏳ {SIGNAL_LIFE_H} ঘণ্টায় SL/TP না ছুঁলে বাতিল ধরবেন। SL/TP ছুঁলে আমি জানাব।`
```

```
            f"⚠️ ট্রেড নেওয়ার সিদ্ধান্ত ও ঝুঁকি আপনার।")

# ===== এক্সিট ট্র্যাকিং =====
def check_hit(s):
    df = klines(s["symbol"], "1m", 1000, start=s["opened"])
    for _, c in df.iterrows():
        if s["side"] == "BUY":
            if c.l <= s["sl"]: return "SL"
            if c.h >= s["tp"]: return "TP"
        else:
            if c.h >= s["sl"]: return "SL"
            if c.l <= s["tp"]: return "TP"
    return None

def track(S):
    now = datetime.now(timezone.utc).timestamp() * 1000
    for s in list(S["open"]):
        try:
            hit = check_hit(s)
        except Exception as e:
            print("track error", e); continue
        name = SYMBOLS[s["symbol"]].split(" ")[0].split("/")[0]
        if hit:
            lvl = s["tp"] if hit == "TP" else s["sl"]
            pct = abs(lvl - s["entry"]) / s["entry"] * 100
            usd = TARGET_USD if hit == "TP" else -RISK_USD
            msg = (f"{'✅ TP ছুঁয়েছে' if hit == 'TP' else '❌ SL ছুঁয়েছে'} — #{s['id']}
                   f"👉 এখন এক্সিট করুন: {fp(lvl)}\n"
                   f"ফলাফল: {'+' if hit == 'TP' else '-'}{pct:.2f}% ≈ {usd:+.2f}$
            s.update({"result": hit, "usd": usd, "closed": datetime.now(timezone.
        elif now - s["opened"] > SIGNAL_LIFE_H * 3600 * 1000:
            msg = f"⌛ #{s['id']} {name} {s['side']} সিগন্যালের সময় শেষ। ট্রেড নিয়ে থাকলে
            s.update({"result": "EXPIRED", "usd": 0.0, "closed": datetime.now(tim
        else:
            continue
        S["open"].remove(s)
        S["history"].append(s)
        if S["chat"]:
            send(S["chat"], msg)

# ===== নতুন সিগন্যাল স্ক্যান =====
def scan(S):
    for sym in SYMBOLS:
        try:
            if any(o["symbol"] == sym for o in S["open"]):
                continue
```

```
            a = analyze(sym)
            if a["side"] and S["last_candle"].get(sym) != a["candle"]:
                S["last_candle"][sym] = a["candle"]
                s = make_signal(a)
                s["id"] = S["next_id"]; S["next_id"] += 1
                S["open"].append(s)
                send(S["chat"], signal_text(s))
        except Exception as e:
            print("scan error", sym, e)

# ===== কমান্ড =====
def cmd_status(S):
    out = []
    for sym, name in SYMBOLS.items():
        try:
            a = analyze(sym)
            p = price(sym)
            tr = {"UP": "ঊর্ধ্বমুখী 📈", "DOWN": "নিম্নমুখী 📉", "FLAT": "সাইডওয়েজ ➖"}[a['
            sig = f"{a['side']} সেটআপ পাওয়া গেছে" if a["side"] else "এখন এন্ট্রির সুযোগ
            out.append(f"{name}\nদাম: {fp(p)}\n1H ট্রেন্ড: {tr}\n15M RSI: {a['rsi']:.0
        except Exception as e:
            out.append(f"{name}: ডেটা আনা যায়নি")
    out.append(f"চলমান সিগন্যাল: {len(S['open'])} | সিগন্যাল: {'⏸️ বন্ধ' if S['paused']
    return "\n\n".join(out)

def cmd_open(S):
    if not S["open"]:
        return "এখন কোনো চলমান সিগন্যাল নেই।"
    out = []
    for s in S["open"]:
        try:
            p = price(s["symbol"])
            d = 1 if s["side"] == "BUY" else -1
            fl = (p - s["entry"]) / s["entry"] * 100 * d
            extra = f"\nএখন {fp(p)} ({fl:+.2f}%)"
        except Exception:
            extra = ""
        out.append(signal_text(s, "📌 চলমান") + extra)
    return "\n\n——\n\n".join(out)

def cmd_stats(S):
    done = [h for h in S["history"] if h.get("result") in ("TP", "SL")]
    if not done:
        return "পরিসংখ্যানের জন্য আগে কিছু সিগন্যাল TP/SL ছুঁতে হবে।"
    w = [h for h in done if h["result"] == "TP"]
    return (f"মোট সিগন্যাল (শেষ): {len(done)}\nজয় (TP): {len(w)} | হার (SL): {len(do
```

```
            f"Win rate: {len(w) / len(done) * 100:.0f}%\n"
            f"নেট ফলাফল: {sum(h['usd'] for h in done):+.2f}$ (প্রতিবার ${RISK_USD:.0

HELP = ("/status — মার্কেটে এখন কী চলছে\n/open — চলমান সিগন্যাল\n/stats — আগের সিগন্যালের 
        "/pause — নতুন সিগন্যাল বন্ধ\n/resume — চালু\n\n"
        "নতুন সিগন্যাল বা SL/TP হলে আমি নিজেই মেসেজ দেব।\n"
        "ℹ️ কমান্ডের উত্তর আসতে কয়েক মিনিট দেরি হতে পারে।")

def handle_updates(S):
    r = tg("getUpdates", offset=S["offset"], timeout=0)
    for u in r.get("result", []):
        S["offset"] = u["update_id"] + 1
        m = u.get("message")
        if not m or "text" not in m:
            continue
        uid, chat = m["from"]["id"], m["chat"]["id"]
        cmd = m["text"].split()[0].split("@")[0].lower()
        if S["owner"] is None and cmd == "/start":
            S["owner"] = uid
        if uid != S["owner"]:
            continue
        S["chat"] = chat
        if cmd == "/start":
            send(chat, "SULTAN WAVE সিগন্যাল বট 🌊\nসুযোগ এলে আমি নিজেই সিগন্যাল পাঠাব (
        elif cmd == "/help":
            send(chat, HELP, KEYBOARD)
        elif cmd == "/status":
            send(chat, cmd_status(S))
        elif cmd == "/open":
            send(chat, cmd_open(S))
        elif cmd == "/stats":
            send(chat, cmd_stats(S))
        elif cmd == "/pause":
            S["paused"] = True; send(chat, "⏸️ নতুন সিগন্যাল বন্ধ। চলমান সিগন্যালের SL/TP
        elif cmd == "/resume":
            S["paused"] = False; send(chat, "▶️ সিগন্যাল চালু।")

def main():
    if not TOKEN:
        sys.exit("BOT_TOKEN নেই")
    S = load()
    handle_updates(S)
    track(S)
    if S["chat"] and not S["paused"]:
        scan(S)
    save(S)
```

```
if __name__ == "__main__":
    main()
```